# Paradigmas de Programação — Aula 09: Modelo de atores com Elixir

## O modelo de atores

## Objetivos da aula

- **Definir** ator — estado privado, caixa de correio, comportamento — e as três ações
- **Contrastar** atores com os canais da aula 08: destino da mensagem e sincronização
- **Explicar** por que não há condição de corrida no estado de um ator, e o que ainda pode dar errado
- **Ler** Elixir básico e **escrever** processos com `spawn`, `send` e `receive`
- **Reescrever** um ator como GenServer e **supervisioná-lo** (*let it crash*)

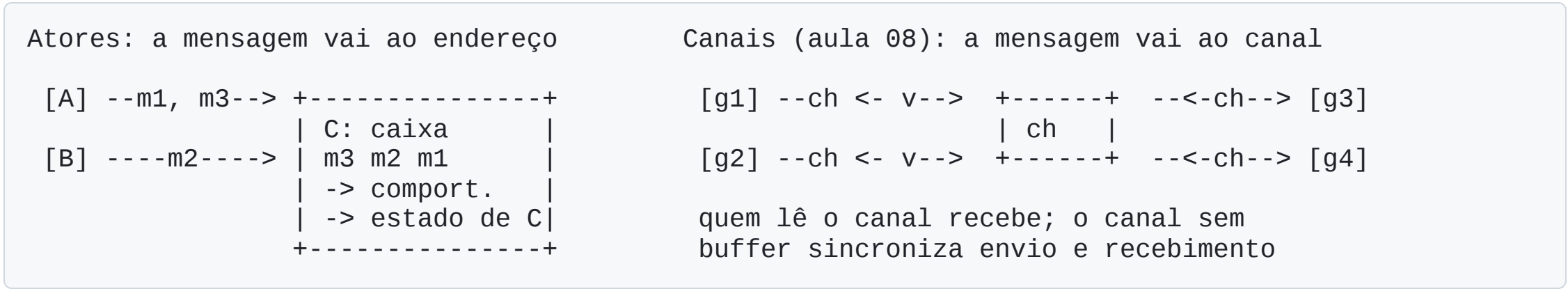
## Ator

- **Ator** (*actor*): unidade de computação com (HEWITT; BISHOP; STEIGER, 1973; AGHA, 1986)
  - **estado privado** — nenhum outro ator lê ou escreve
  - **caixa de correio** (*mailbox*) — fila de mensagens recebidas e não tratadas
  - **comportamento** — o que fazer com cada mensagem, dado o estado
- Todo ator tem um **endereço**; a mensagem é o único meio de interação
- Ao tratar uma mensagem, o ator pode (AGHA, 1986):
  i. **enviar** um número finito de mensagens a endereços que conhece
  ii. **criar** um número finito de novos atores
  iii. **definir o comportamento** (e o estado) para a próxima mensagem
- $b(s, m) = (\text{envios},\ \text{criados},\ s')$ — o contador em 5 que recebe `incrementar` passa a se comportar como contador em 6

## Uma mensagem por vez: sem condição de corrida

- O ator trata as mensagens **uma de cada vez**; atores distintos executam concorrentemente entre si
- Aula 08: `contador++` em duas threads = ler, somar, escrever **intercalados** → incremento perdido → trava
- Com atores: o contador é estado de um ator, cada incremento é uma mensagem
  - duas mensagens `incrementar` simultâneas entram na caixa em alguma ordem e são tratadas **em sequência**
  - sem intercalação dentro do ator: sem seção crítica, sem trava
- *Check-then-act* (saque, lance) também fica correto: comparar e escrever ocorrem no tratamento de **uma** mensagem
- A exclusão mútua não some: passa a ser garantida pela construção, e não por uma trava em cada acesso

## Atores × canais




| | Canais (Go) | Atores |
|---|---|---|
| Destino | um canal, independente das gorrotinas | um ator, pelo endereço |
| Sincronização | sem *buffer*: remetente bloqueia até o recebimento | nenhuma: deposita e segue |
| Estado | na gorrotina dona, **por convenção** | dentro do ator, **por construção** |
| Fila | do canal, tamanho fixo (0 ou `n`) | do ator, sem limite |

## Garantias e limites

- **Ordem**: o modelo teórico garante entrega, não ordem; a BEAM garante ordem **só entre o mesmo par remetente → destinatário**
  - `m1` antes de `m3` (ambas de A); `m2` pode cair em qualquer posição
  - $k$ remetentes com $n_1, \ldots, n_k$ mensagens: $\frac{(n_1+\cdots+n_k)!}{n_1!\cdots n_k!}$ ordens de chegada — no exemplo, $3!/(2!\,1!) = 3$
- **Caixa sem limite**: o envio nunca bloqueia; ator mais lento que os remetentes acumula mensagens até esgotar a memória — sem *backpressure*, ao contrário do canal com *buffer*
- **A resposta é outra mensagem**: o envio não devolve valor; o remetente envia o **próprio endereço** — `{saldo, endereço_do_remetente}` — e o destinatário responde a ele
  - quem espera resposta depende de que ela chegue: na prática, espera com prazo